CHAPTER

FIFTEEN

SYSTEM ADDRESS MAP INTERFACES

This section explains how an ACPI-compatible system conveys its memory resources/type mappings to OSPM. There are three ways for the system to convey memory resources /mappings to OSPM. The first is an INT 15 BIOS interface that is used in IA-PC-based systems to convey the system's initial memory map. UEFI enabled systems use the UEFI GetMemoryMap() boot services function to convey memory resources to the OS loader. These resources must then be conveyed by the OS loader to OSPM. See the UEFI Specification for more information on UEFI services.

Lastly, if memory resources may be added or removed dynamically, memory devices are defined in the ACPI Namespace conveying the resource information described by the memory device (see *Memory Devices*).

ACPI defines the following address range types.

Table 15.1: **Address Range Types**

Value	Mnemonic	Save in S4	Description
1	AddressRangeMemory	Yes	This range is available RAM usable by the operating system.
2	AddressRangeReserved	No	This range of addresses is in use or reserved by the system and is not to be included in the allocatable memory pool of the operating system's memory manager.
3	AddressRangeACPI	Yes	ACPI Reclaim Memory. This range is available RAM usable by the OS after it reads the ACPI tables.
4	AddressRangeNVS	Yes	ACPI NVS Memory. This range of addresses is in use or reserved by the system and must not be used by the operating system. This range is required to be saved and restored across an NVS sleep.
5	AddressRangeUnusable	No	This range of addresses contains memory in which errors have been detected. This range must not be used by OSPM.
6	AddressRangeDisabled	No	This range of addresses contains memory that is not enabled. This range must not be used by OSPM.

continues on next page

Table 15.1 – continued from previous page

Value	Mnemonic	Save in S4	Description
7	AddressRangePersistent-Memory	No	OSPM must comprehend this memory as having non-volatile attributes and handle distinct from conventional volatile memory. The memory region supports byte-addressable non-volatility. NOTE: Extended Attributes for the memory reported using AddressRangePersistentMemory should set Bit [0] to 1 (see *Extended Attributes for Address Range Descriptor Structure*).
8	AddressRangeUnaccepted	No	A memory region that represents unaccepted memory, that must be accepted by the boot target before it can be used. Unless otherwise noted, all other memory types are accepted. For platforms that support unaccepted memory, all unaccepted valid memory will be reported as unaccepted in the memory map. Unreported physical address ranges must be treated as not-present memory.
9-11	Undefined	No	Reserved for future use. OSPM must treat any range of this type as if the type returned was *AddressRangeReserved* .
12	OEM defined	No	An OS should not use a memory type in the vendor-defined range because collisions may occur between different vendors.
13 to 0xEFFFFFFF	Undefined	No	Reserved for future use. OSPM must treat any range of this type as if the type returned was *AddressRangeReserved* .
0xF0000000 to 0xFFFFFFFF	OEM defined	No	An OS should not use a memory type in the vendor-defined range because collisions may occur between different vendors.

Platform runtime firmware can use the AddressRangeReserved address range type to block out various addresses as not suitable for use by a programmable device. Some of the reasons a platform runtime firmware would do this are:

- The address range contains system ROM.
- The address range contains RAM in use by the ROM.
- The address range is in use by a memory-mapped system device.
- The address range is, for whatever reason, unsuitable for a standard device to use as a device memory space.
- The address range is within an NVRAM device where reads and writes to memory locations are no longer successful, that is, the device was worn out.
- OSPM will not save or restore memory reported as AddressRangeReserved, AddressRangeUnusable, AddressRangeDisabled, or AddressRangePersistentMemory when transitioning to or from the S4 sleeping state.
- Platform boot firmware must ensure that contents of memory that is reported as AddressRangePersistentMemory is retained after a system reset or a power cycle event.

15.1 INT 15H, E820H - Query System Address Map

This interface is used in real mode only on IA-PC-based systems and provides a memory map for all of the installed RAM, and of physical memory ranges reserved by the BIOS. The address map is returned through successive invocations of this interface; each returning information on a single range of physical addresses. Each range includes a type that indicates how the range of physical addresses is to be treated by the OSPM.

If the information returned from E820 in some way differs from INT-15 88 or INT-15 E801, the information returned from E820 supersedes the information returned from INT-15 88 or INT-15 E801. This replacement allows the BIOS to return any information that it requires from INT-15 88 or INT-15 E801 for compatibility reasons. For compatibility reasons, if E820 returns any AddressRangeACPI or AddressRangeNVS memory ranges below 16 MiB, the INT-15 88 and INT-15 E801 functions must return the top of memory below the AddressRangeACPI and AddressRangeNVS memory ranges.

The memory map conveyed by this interface is not required to reflect any changes in available physical memory that have occurred after the BIOS has initially passed control to the operating system. For example, if memory is added dynamically, this interface is not required to reflect the new system memory configuration.

Table 15.2: **Input to the INT 15h E820h Call**

Register	Contents	Description
EAX	Function Code	E820h
EBX	Continuation	Contains the continuation value to get the next range of physical memory. This is the value returned by a previous call to this routine. If this is the first call, EBX must contain zero.
ES:DI	Buffer Pointer	Pointer to an Address Range Descriptor structure that the BIOS fills in.
ECX	Buffer Size	The length in bytes of the structure passed to the BIOS. The BIOS fills in the number of bytes of the structure indicated in the ECX register, maximum, or whatever amount of the structure the BIOS implements. The minimum size that must be supported by both the BIOS and the caller is 20 bytes. Future implementations might extend this structure.
EDX	Signature	'SMAP' Used by the BIOS to verify the caller is requesting the system map information to be returned in ES:DI.

Table 15.3: **Output from the INT 15h E820h Call**

Register	Contents	Description
CF	Carry Flag	Non-Carry - Indicates No Error
EAX	Signature	'SMAP.' Signature to verify correct BIOS revision.
ES:DI	Buffer Pointer	Returned Address Range Descriptor pointer. Same value as on input.
ECX	Buffer Size	Number of bytes returned by the BIOS in the address range descriptor. The minimum size structure returned by the BIOS is 20 bytes.
EBX	Continuation	Contains the continuation value to get the next address range descriptor. The actual significance of the continuation value is up to the discretion of the BIOS. The caller must pass the continuation value unchanged as input to the next iteration of the E820 call in order to get the next Address Range Descriptor. A return value of zero means that this is the last descriptor. Note: the BIOS can also indicate that the last descriptor has already been returned during previous iterations by returning the carry flag set. The caller will ignore any other information returned by the BIOS when the carry flag is set.

Table 15.4: **Address Range Descriptor Structure**

Offset in Bytes	Name	Description
0	BaseAddrLow	Low 32 Bits of Base Address
4	BaseAddrHigh	High 32 Bits of Base Address
8	LengthLow	Low 32 Bits of Length in Bytes
12	LengthHigh	High 32 Bits of Length in Bytes
16	Type	Address type of this range
20	Extended Attributes	See the *Extended Attributes for Address Range Descriptor Structure*

The BaseAddrLow and BaseAddrHigh together are the 64-bit base address of this range. The base address is the physical address of the start of the range being specified.

The LengthLow and LengthHigh together are the 64-bit length of this range. The length is the physical contiguous length in bytes of a range being specified.

The Type field describes the usage of the described address range as defined in *Address Range Types*.

Table 15.5: **Extended Attributes for Address Range Descriptor Structure**

Bit	Mnemonic	Description
0	*Reserved*	Reserved, must be set to 1.
2:1	Reserved	Reserved, must be set to 0.
3	AddressRangeErrorLog	If set, the address range descriptor represents memory used for logging hardware errors.
31:4	*Reserved*	Reserved for future use.

Note

Bit [1] and [2] above were deprecated as of ACPI 6.1. Bit [3] is used only on PC-AT BIOS systems to pinpoint the error log in memory. On UEFI-based systems, either UEFI Hardware Error Record HwErrRec#### runtime UEFI variable interface or the Error Record Serialization Actions 0xD, 0xE and 0xF for the APEI ERST interface must be implemented for the error logs.

15.2 E820 Assumptions and Limitations

- The platform boot firmware returns address ranges describing baseboard memory.
- The platform boot firmware does not return a range description for the memory mapping of PCI devices, ISA Option ROMs, and ISA Plug and Play cards because the OS has mechanisms available to detect them.
- The platform boot firmware returns chip set-defined address holes that are not being used by devices as reserved.
- Address ranges defined for baseboard memory-mapped I/O devices, such as APICs, are returned as reserved.
- All occurrences of the system platform boot firmware are mapped as reserved, including the areas below 1 MB, at 16 MB (if present), and at end of the 4-GB address space.

- Standard PC address ranges are not reported. For example, video memory at A0000 to BFFFF physical addresses are not described by this function. The range from E0000 to EFFFF is specific to the baseboard and is reported as it applies to that baseboard.
- All of lower memory is reported as normal memory. The OS must handle standard RAM locations that are reserved for specific uses, such as the interrupt vector table (0:0) and the platform boot firmware data area (40:0).

15.3 UEFI GetMemoryMap() Boot Services Function

EFI enabled systems use the UEFI *GetMemoryMap()* boot services function to convey memory resources to the OS loader. These resources must then be conveyed by the OS loader to OSPM.

The GetMemoryMap interface is only available at boot services time. It is not available as a run-time service after OSPM is loaded. The OS or its loader initiates the transition from boot services to run-time services by calling *ExitBootServices()* . After the call to *ExitBootServices()* all system memory map information must be derived from objects in the ACPI Namespace.

The *GetMemoryMap()* interface returns an array of UEFI memory descriptors. These memory descriptors define a system memory map of all the installed RAM, and of physical memory ranges reserved by the firmware. Each descriptor contains a type field that dictates how the physical address range is to be treated by the operating system. The table below defines the mapping from UEFI memory types (see UEFI Specification) to ACPI *Address Range Types* that:

- Platform boot firmware shall follow if describing the memory range in both UEFI and legacy BIOS modes; and
- aAn OS loader should use if it conveys that information to the OS using an ACPI E820h system address map table.

Table 15.6: **UEFI Memory Types and mapping to ACPI address range types**

Type	Mnemonic	ACPI Address Range Type
0	EfiReservedMemoryType	AddressRangeReserved
1	EfiLoaderCode	AddressRangeMemory
2	EfiLoaderData	AddressRangeMemory
3	EfiBootServicesCode	AddressRangeMemory
4	EfiBootServicesData	AddressRangeMemory
5	EfiRuntimeServiceCode	AddressRangeReserved
6	EfiRuntimeServicesData	AddressRangeReserved
7	EfiConventionalMemory	AddressRangeMemory
8	EfiUnusableMemory	AddressRangeReserved
9	EfiACPIReclaimMemory	AddressRangeACPI
10	EfiACPIMemoryNVS	AddressRangeNVS
11	EfiMemoryMappedIO	AddressRangeReserved
12	EfiMemoryMappedIOPortSpace	AddressRangeReserved
13	EfiPalCode	AddressRangeReserved
14	EfiPersistentMemory	AddressRangePersistentMemory
15 to 0x6FFFFFFF	Reserved.	AddressRangeReserved
0x70000000 to 0x7FFFFFFF	Reserved for OEM used	An OS should not use a memory type in the vendor-defined range because collisions may occur between different vendors.
0x80000000 to 0xFFFFFFFF	Reserved for use by UEFI OS loaders that are provided by operating system vendors	OSV defined

The table above applies to system firmware that supports legacy BIOS mode plus UEFI mode, and OS loaders.

15.4 UEFI Assumptions and Limitations

- The firmware returns address ranges describing the current system memory configuration.
- The firmware does not return a range description for the memory mapping of PCI devices, ISA Option ROMs, and ISA Plug and Play cards because the OS has mechanisms available to detect them.
- The firmware does not return a range description for address space regions that are not backed by physical hardware except those mentioned above. Regions that are backed by physical hardware, but are not supposed to be accessed by the OS, must be returned as reserved. Herein 'reserved' is the definition of the term as noted by the ACPI specification as ACPI address range reserved. OS may use addresses of memory ranges that are not described in the memory map at its own discretion
- Address ranges defined for baseboard memory-mapped I/O devices, such as APICs, are returned as reserved.
- All occurrences of the system firmware are mapped as reserved, including the areas below 1 MB, at 16 MB (if present), and at end of the 4-GB address space.
- Standard PC address ranges are not reported. For example, video memory at A0000 to BFFFF physical addresses are not described by this function. The range from E0000 to EFFFF is specific to the baseboard and is reported as it applies to that baseboard.
- All of lower memory is reported as normal memory. The OS must handle standard RAM locations that are reserved for specific uses, such as the interrupt vector table (0:0) and the platform boot firmware data area (40:0). To preserve backward compatibility, platform should avoid using persistent memory to materialize the lower memory. If persistent memory is used for lower memory, platform boot firmware must report the lower memory address range using AddressRangeMemory and must not report using AddressRangePersistentMemory.
- EFI contains descriptors for memory mapped I/O and memory mapped I/O port space to allow for virtual mode calls to UEFI run-time functions. The OS must never use these regions.

15.5 Example Address Map

This sample address map (for an Intel processor-based system) describes a machine that has 128 MiB of RAM, 640 KiB of base memory and 127 MiB of extended memory. The base memory has 639 KiB available for the user and 1 KiB for an extended BIOS data area. A 4-MiB Linear Frame Buffer (LFB) is based at 12 MiB. The memory hole created by the chip set is from 8 MiB to 16 MiB. Memory-mapped APIC devices are in the system. The I/O Unit is at FEC00000 and the Local Unit is at FEE00000. The system BIOS is remapped to 1 GB-64 KiB.

The 639-KiB endpoint of the first memory range is also the base memory size reported in the BIOS data segment at 40:13. The following table shows the memory map of a typical system.

Table 15.7: **Sample Memory Map**

Base (Hex)	Length	Type	Description
0000 0000	639 KiB	AddressRangeMemory	Available Base memory. Typically the same value as is returned using the INT 12 function.
0009 FC00	1 KiB	AddressRangeReserved	Memory reserved for use by the BIOS(s). This area typically includes the Extended BIOS data area.
000F 0000	64 KiB	AddressRangeReserved	System BIOS
0010 0000	7 MiB	AddressRangeMemory	Extended memory, which is not limited to the 64-MiB address range.
0080 0000	4 MiB	AddressRangeReserved	Chip set memory hole required to support the LFB mapping at 12 MiB.
0100 0000	60 MiB	AddressRangeMemory	Baseboard RAM relocated above a chip set memory hole.
04C0 0000	60 MiB	AddressRangePersistent-Memory	Persistent memory that has non-volatile attributes located in this region.
FEC0 0000	4 KiB	AddressRangeReserved	I/O APIC memory mapped I/O at FEC00000.
FEE0 0000	4 KiB	AddressRangeReserved	Local APIC memory mapped I/O at FEE00000.
FFFF 0000	64 KiB	AddressRangeReserved	Remapped System BIOS at end of address space.

15.6 Example: Operating System Usage

The following code segment illustrates the algorithm to be used when calling the Query System Address Map function. This is an implementation example and uses non-standard mechanisms:

```
E820Present = FALSE;
Reg.ebx = 0;
do {
  Reg.eax = 0xE820;
  Reg.es = SEGMENT (&Descriptor);
  Reg.di = OFFSET (&Descriptor);
  Reg.ecx = sizeof (Descriptor);
  Reg.edx = 'SMAP';

   \_int( 15, regs );

  if ((Regs.eflags & EFLAG_CARRY) \|\| Regs.eax != 'SMAP') {
    break;
  }

  if (Regs.ecx < 20 \|\| reg.ecx > sizeof (Descriptor) ) {
    // bug in bios - all returned descriptors must be
    // at least 20 bytes long, and cannot be larger then
    // the input buffer.
    break;
  }

  E820Present = TRUE;
  .
  .
  .
  Add address range Descriptor.BaseAddress through
  Descriptor.BaseAddress + Descriptor.Length
  as type Descriptor.Type
  .
  .
  .

} while (Regs.ebx != 0);

if (!E820Present) {
  .
  .
  .
  call INT-15 88 and/or INT-15 E801 to obtain old style memory information
  .
  .
  .
}
```